

32-bit Arithmetic Logic Unit (ALU)

Design and Verification using Verilog HDL

Internship Project Report

Prepared by

Ramkrupakiran Akkarapaka

B.Tech – Electronics and Communication Engineering

Atria Institute of Technology

Bengaluru, India

Table of Contents

1. Introduction
2. Objectives
3. Software Tools Used
4. ALU Architecture
5. RTL Design
6. Supported Operations
7. Testbench Design
8. Simulation Results
9. Waveform Analysis
10. Conclusion
11. Future Scope

1. Introduction

Overview:

The **Arithmetic Logic Unit (ALU)** is one of the fundamental components of every digital processor. It is responsible for performing arithmetic and logical operations on binary data, enabling processors to execute mathematical calculations and make logical decisions. Operations such as addition, subtraction, bitwise logic, shifting, and comparison are all carried out by the ALU.

In this project, a **32-bit Arithmetic Logic Unit (ALU)** was designed and implemented using **Verilog Hardware Description Language (HDL)**. The ALU supports nine operations, including addition, subtraction, bitwise AND, OR, XOR, NOT, left shift, right shift, and comparison. A dedicated Verilog testbench was developed to verify the correctness of each operation through simulation.

The design was compiled and simulated using **Icarus Verilog**, while the generated waveforms were analyzed using a waveform viewer. Throughout this project, the complete RTL design workflow was followed, including hardware description, functional verification, simulation, debugging, and documentation.

This project provided practical experience in digital hardware design and verification while demonstrating the implementation of a combinational logic circuit using Verilog HDL.

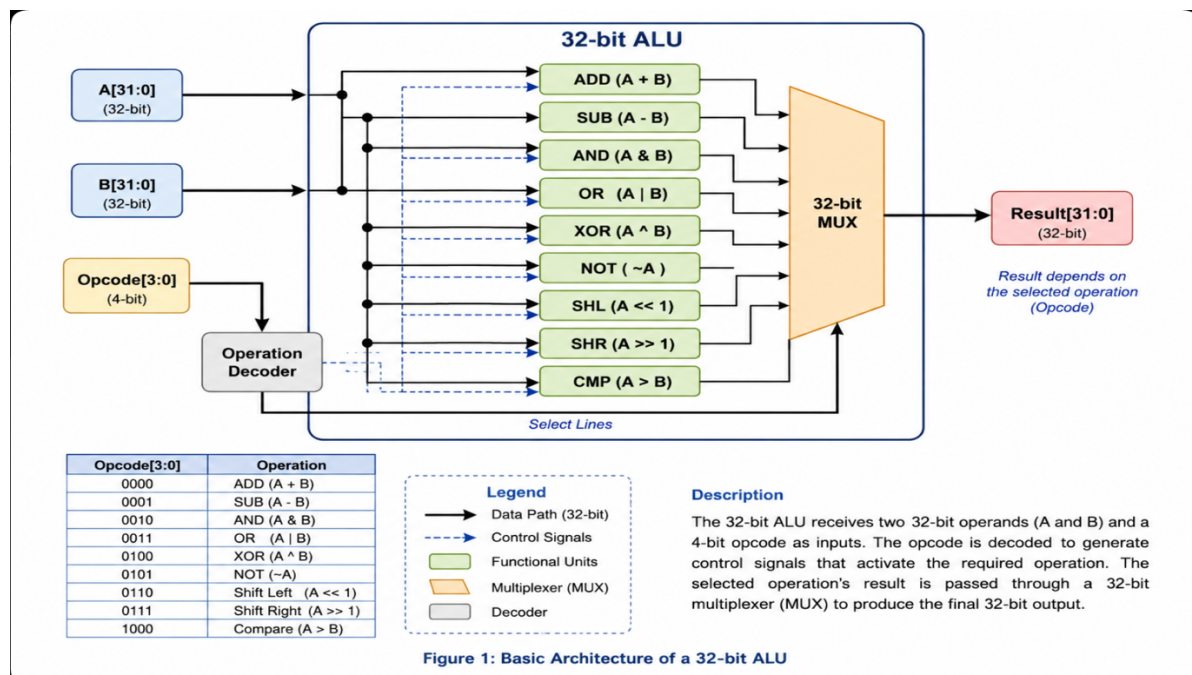


Figure 1: Basic Architecture of a 32-bit ALU

Figure 1: Basic Architecture of the 32-bit Arithmetic Logic Unit

2.Objectives

- Design a 32-bit ALU using Verilog HDL.
- Implement arithmetic, logical, shift, and comparison operations.
- Develop a Verilog testbench for verification.
- Simulate the design using Icarus Verilog.
- Validate outputs using waveform analysis.

3. Software Tools Used

Software	Purpose
Visual Studio Code	Writing and editing Verilog source code
Verilog HDL	Hardware Description Language used to design the ALU
Icarus Verilog	Compilation and simulation of the Verilog design
VS Code Waveform Viewer	Viewing and analyzing simulation waveforms
Git & GitHub	Version control and project hosting

The ALU was implemented using Verilog HDL in Visual Studio Code. Functional verification was performed using a custom testbench. Icarus Verilog was used to compile and simulate the design, while the generated VCD waveform was analyzed using a waveform viewer. Git and GitHub were used to organize and manage the project repository

4.ALU Architecture:

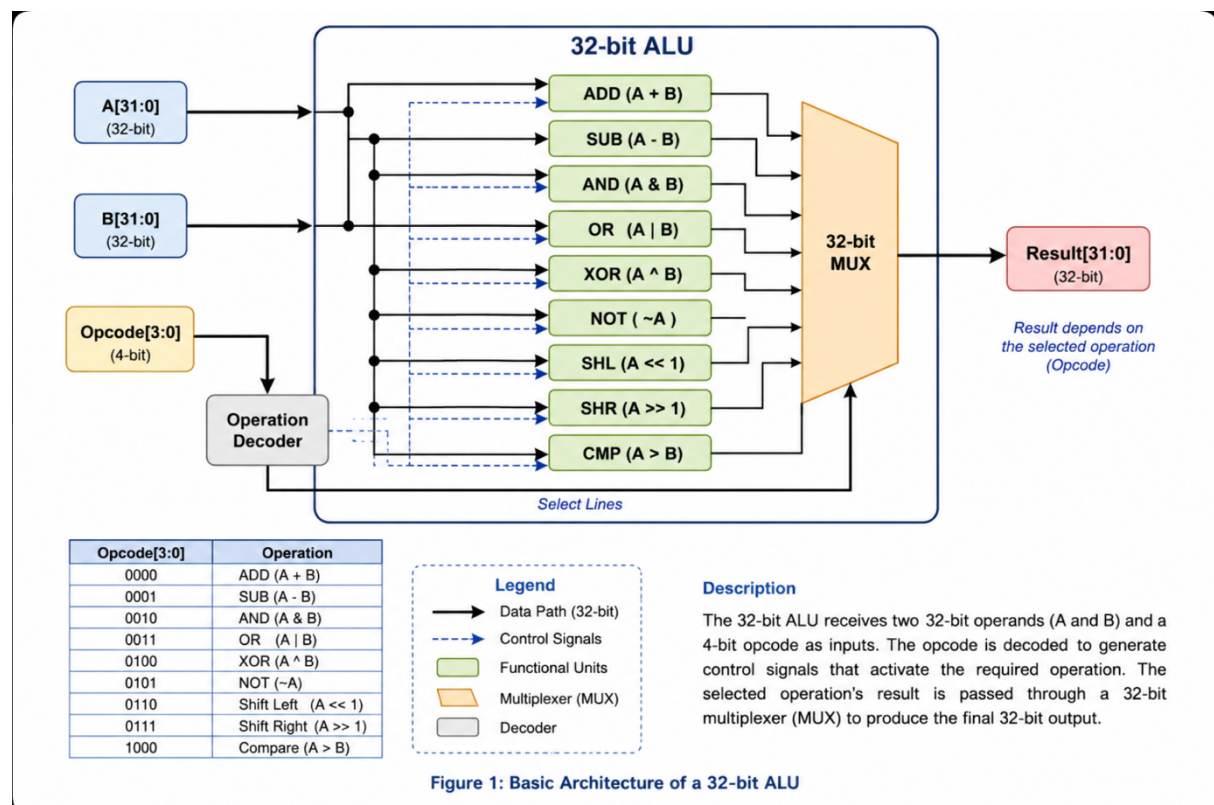


Figure 1: Basic Architecture of the 32-bit Arithmetic Logic Unit:

The ALU accepts two 32-bit operands (A and B) and a 4-bit opcode. The opcode determines which functional unit performs the required operation. Arithmetic, logical, shift, and comparison units operate on the inputs, and a multiplexer selects the appropriate output based on the opcode. The selected result is presented as the 32-bit output.

5.RTL Design

5.1 Module Declaration

```
module alu32(  
    input [31:0] A,  
    input [31:0] B,  
    input [3:0] opcode,  
    output reg [31:0] result  
);
```

The ALU is implemented as a Verilog module named `alu32`. It accepts two 32-bit input operands (A and B), a 4-bit opcode, and produces a 32-bit output (result). Since the output is assigned inside an `always` block, it is declared as a `reg` type.

5.2 Combinational Logic

```
always @(*) begin
```

The `always @(*)` block represents combinational logic. It ensures that whenever any input signal changes, the ALU immediately recomputes the output without waiting for a clock edge.

5.3 Opcode Selection

```
case(opcode)
```

The `case` statement selects the required operation based on the 4-bit opcode. Each opcode corresponds to a unique arithmetic, logical, shift, or comparison operation.

5.4 Example Operations

```
4'b0000: result = A + B;  
4'b0001: result = A - B;  
4'b0010: result = A & B;
```

Similar case statements are implemented for OR, XOR, NOT, shift left, shift right, and comparison operations. A default case assigns zero to the output for unsupported opcodes, ensuring predictable behavior.

6. Supported Operations

Opcode	Operation	Description
0000	Addition	Adds A and B
0001	Subtraction	Subtracts B from A
0010	AND	Bitwise AND
0011	OR	Bitwise OR
0100	XOR	Bitwise XOR
0101	NOT	Bitwise inversion of A
0110	Shift Left	Shifts A left by one bit
0111	Shift Right	Shifts A right by one bit
1000	Compare	Returns 1 if $A > B$, otherwise 0

7. Testbench Design

Code Snippet

```
initial begin
    $monitor("Time=%0t A=%d B=%d opcode=%b result=%d",
            $time, A, B, opcode, result);
end
initial begin
    A = 32'd10;
    B = 32'd15;
    opcode = 4'b0000;
    #20;

    A = 32'd50;
    B = 32'd23;
    opcode = 4'b0001;
    #20;

    A = 32'd29;
    B = 32'd25;
    opcode = 4'b0010;
    #20;
    $finish;
end
```

A Verilog testbench was developed to verify the functionality of the 32-bit ALU. The testbench applies different input combinations and opcode values to validate each supported operation. The \$monitor system task continuously displays the values of all important signals whenever they change during simulation. After executing all test cases, the simulation is terminated using \$finish.

8. Simulation Results

```
Time=0
A=10 B=15 opcode=0000 result=25
```

```
Time=20000
A=50 B=23 opcode=0001 result=27
```

```
Time=40000
A=29 B=25 opcode=0010 result=25
```

```
...
```

```
Time=160000
A=60 B=40 opcode=1000 result=1
```

The simulation results confirm that each ALU operation produced the expected output. Arithmetic, logical, shift, and comparison operations were successfully verified using the developed testbench.

9. Waveform Analysis

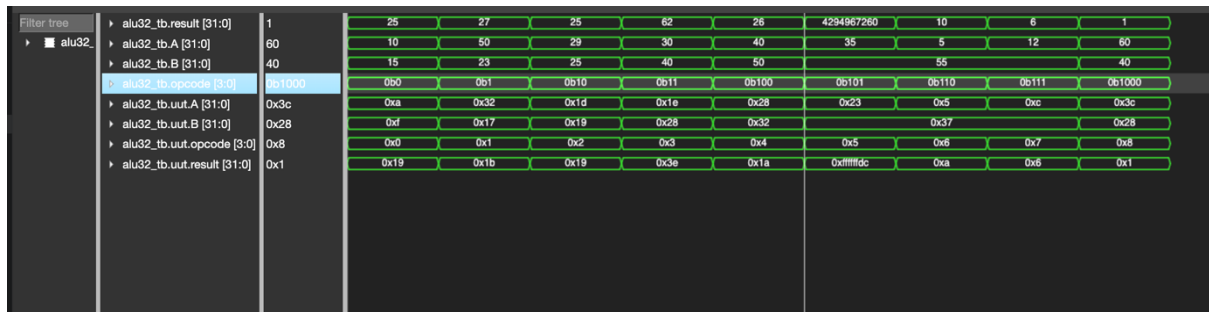


Figure 2: Simulation Waveform of the 32-bit ALU

The waveform verifies that the ALU responds correctly to changes in the input operands and opcode. Every 20 ns, the testbench applies a new test vector, causing the output result to update immediately because the ALU is implemented as combinational logic using the `always @(*)` block.

10. Conclusion

A 32-bit Arithmetic Logic Unit was successfully designed and verified using Verilog HDL. The implementation supports arithmetic, logical, shift, and comparison operations. Functional verification was performed using a custom Verilog testbench, and all operations produced the expected outputs during simulation. The project provided practical experience in RTL design, functional verification, simulation, and debugging using industry-standard digital design tools.

11. Future Scope

- Implement overflow detection.
- Add carry, zero, sign, and overflow flags.
- Support multiplication and division operations.
- Design a pipelined ALU.
- Integrate the ALU into a simple processor datapath.
- Develop a self-checking testbench with automated PASS/FAIL reporting.

12. References

1. Verilog HDL: A Guide to Digital Design and Synthesis – Samir Palnitkar
2. Icarus Verilog Documentation
3. IEEE Standard for Verilog Hardware Description Language (IEEE 1364)